So, what's an iterator? Think of it as an object with a `next` method:

```
keyCommandIterator =
  next: ->
    if Math.floor Math.random()*10 > 5
      UP
    else
      DOWN

keyCommandIterator.next()
# UP
keyCommandIterator.next()
# UP
```

An iterator that uses Math.random() to return either UP or DOWN

To use this iterator in ECMAScript 6, you need to declare it as an `iterator` property on the prototype of your object. In CoffeeScript, that means putting it in the class declaration:

```
class KeyboardEvents
  iterator: keyCommandIterator
```

You iterate over an instance of `KeyboardEvents` with an ECMAScript 6 comprehension:

```
scruffysKeyboard = new KeyboardEvents
for event of scruffysKeyboard
  console.log scruffysKeyboard

# UP
# UP
# DOWN
```

This will run until you terminate it.

Now the potential for comprehensions starts to become apparent, but if you have a sweet tooth, it might not seem like very much sugar yet. Just making comprehensions work with objects that have a `next` method isn't enough. That's good, because it doesn't end there. Comprehensions in ECMAScript 6 will also work with a new sort of function: a generator.

Generators